Natural Language Processing

Session 08 – The BERT Ecosystem

Prof. Dr. Richard Sieg
SoSe 2026

Exam for DIS Bachelor Students

- The exams are in groups of three (i.e. your team)
- There are three broad categories text processing & statistics, text classification (including BERT), and LLMs
- You distribute the categories among the team
- Each team member starts with presenting ***one completely new approach*** within their category which was not discussed during the lecture and exercises. For example
  - A new statistic and visualization
  - A new classification (including the evaluation)
  - A new use of LLMs

Exam for DIS Bachelor Students

- The presentation takes around 7 minutes per team member including follow-up questions
- After each presentation, the team member is then asked two random questions from a question pool about their implementation of the assignments in the other two categories. ***The team member has to show that they understand the NLP theory behind this implementation.***
  - So, make sure that your marimo notebooks are up and running
- Grading follows the following rubric

NLP Oral Exam Grading Rubric — DIS Bachelor (SoSe 2026)

Group exam (teams of 3), approx. 30 min per team. Each member: presentation of a new approach (5–7 min incl. follow-up) + 2 random questions from the other categories. **A score of 0 in any Part 1 criterion → automatic fail (5.0).**

Criterion	3	2	1	0	Points
Part 1: Presentation of a New Approach (60%)					
1.1 Technical Quality of the Implementation (25%)	Implementation is correct, well-structured, and demonstrates a solid understanding of the underlying NLP concepts. The student can explain why this approach is appropriate and what its limitations are.	Implementation works but has some issues. The student explains the main concepts but struggles with details or has small gaps.	Implementation is incomplete or has significant errors. The student has difficulty explaining the approach. The approach is barely distinguishable from what was covered in class.	Implementation does not work. The student cannot explain what they did or why.	
1.2 Presentation & Live Demo (15%)	Clear, well-structured presentation with a live code demonstration. The marimo notebook runs smoothly. Timing is appropriate (5–7 min). The student speaks freely and confidently.	Understandable presentation, demo works with minor hiccups. Minor timing issues (±1–2 min). The student may rely on notes but can explain the content.	Hard to follow, demo has significant issues or does not run. Poor time management (< 4 min or > 9 min). The student reads from notes.	Presentation is incoherent or missing. No working demo. Unprepared.	
1.3 Follow-Up Questions on the Presentation (20%)	Answers follow-up questions precisely. Demonstrates deep understanding and can reason about alternatives, trade-offs, and edge cases.	Answers most questions adequately but stays at the surface level. Can reason about the approach when prompted.	Struggles to answer. Gives vague or evasive responses.	Cannot answer follow-up questions. Shows no understanding beyond what was presented.	
Part 2: Theory Questions on the Other Two Categories (40%)					
Question 1					
2.1 Theoretical Understanding (15%)	Explains the NLP theory behind the question confidently and correctly. Connects concepts across sessions.	Explains the basics but has difficulty with details or connecting concepts. Needs prompting to get to the right answer.	Shows only superficial understanding. Significant gaps in theoretical knowledge.	Cannot explain the NLP theory. Shows no understanding of the concepts.	
2.2 Practical Demonstration (5%)	Navigates the marimo notebook confidently. Can quickly find and explain the relevant code. Demonstrates that they actively worked on this assignment.	Can locate code but needs time. Explains what the code does but not always why.	Has difficulty finding or explaining the code. Appears unfamiliar with this part of the team's work.	Cannot navigate or explain the code. Suggests they did not participate in the assignment.	
Question 2					
2.3 Theoretical Understanding (15%)	Same criteria as 2.1.	Same criteria as 2.1.	Same criteria as 2.1.	Same criteria as 2.1.	
2.4 Practical Demonstration (5%)	Same criteria as 2.2.	Same criteria as 2.2.	Same criteria as 2.2.	Same criteria as 2.2.	

Grade Calculation

Total = (1.1 × 0.25) + (1.2 × 0.15) + (1.3 × 0.20) + (2.1 × 0.15) + (2.2 × 0.05) + (2.3 × 0.15) + (2.4 × 0.05)

Points	≥ 2.7	≥ 2.5	≥ 2.3	≥ 2.1	≥ 2.0	≥ 1.9	≥ 1.8	≥ 1.7	≥ 1.6	≥ 1.5	< 1.5
Grade	1.0	1.3	1.7	2.0	2.3	2.7	3.0	3.3	3.7	4.0	5.0

Student: _____ Team: _____ Date: _____ Total Score: _____ Grade: _____

Schedule

Date	Weekday	CW	Room	Topic
16.04.	Thursday	16	149	Introduction: The World of NLP
23.04.	Thursday	17	149	Text Processing Fundamentals
30.04.	Thursday	18	149	Linguistic Annotations & Pattern Matching
05.05.	Tuesday	19	154	Text Vectorization: From Text to Numbers ⚠️ Substitute for 14.05.
07.05.	Thursday	19	149	Text Classification
28.05.	Thursday	22	149	Language Models, Word Vectors + 🎤 Guest Lecture Ines Montani
01.06.	Monday	23	390	📝 **Exams — Master DS Students**
02.06.	Tuesday	23	154	BERT: Pre-Training & Fine-Tuning ⚠️ Substitute for 04.06.
11.06.	Thursday	24	149	The BERT Ecosystem
18.06.	Thursday	25	149	Introduction to LLMs
25.06.	Thursday	26	149	Working with LLMs
02.07.	Thursday	27	149	Ethics, Limitations & Exam Preparation
09.07.	Thursday	28	149	📝 Exams (afternoon) — Bachelor DIS Students
10.07.	Friday	28	149	📝 Exams (afternoon) — Bachelor DIS Students

Agenda

00. Repeat BERT Tutorial

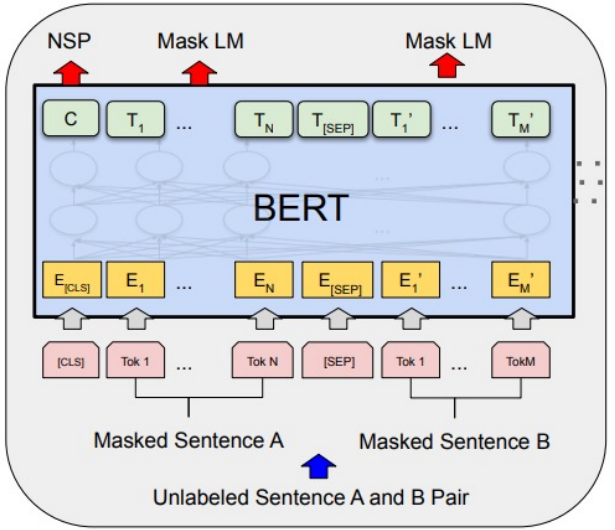
01. SentenceBERT

02. Semantic Search

03. Topic Modeling with BERTopic

What BERT Gives Us

- BERT is a bidirectional transformer encoder: every word sees the whole sentence
- It is pre-trained on huge amounts of raw text (masked language modeling), then fine-tuned for a task
- Its output is a contextual embedding for every token: one vector per word, shaped by the words around it
- So far, we used these token vectors for:
  - Classification (read off the [CLS] token, add a small head)
  - Sequence labeling like NER (one label per token)

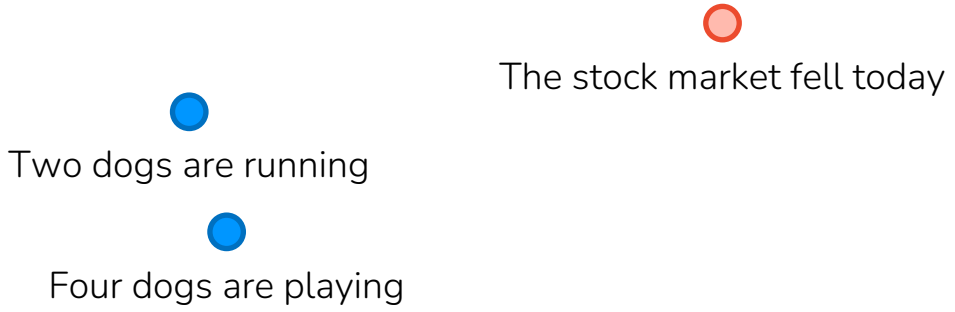


Pre-training

One Vector for a Whole Text

- BERT gives us a vector per word. But very often we want a single vector for a whole sentence or document, so we can compare texts to each other.
- Search: which document best matches this query?
- Clustering / topics: which reviews are about the same thing?
- Deduplication: are these two support tickets the same issue?
- Recommendation: find articles similar to this one

Take two texts, get one number for how similar they are in meaning.

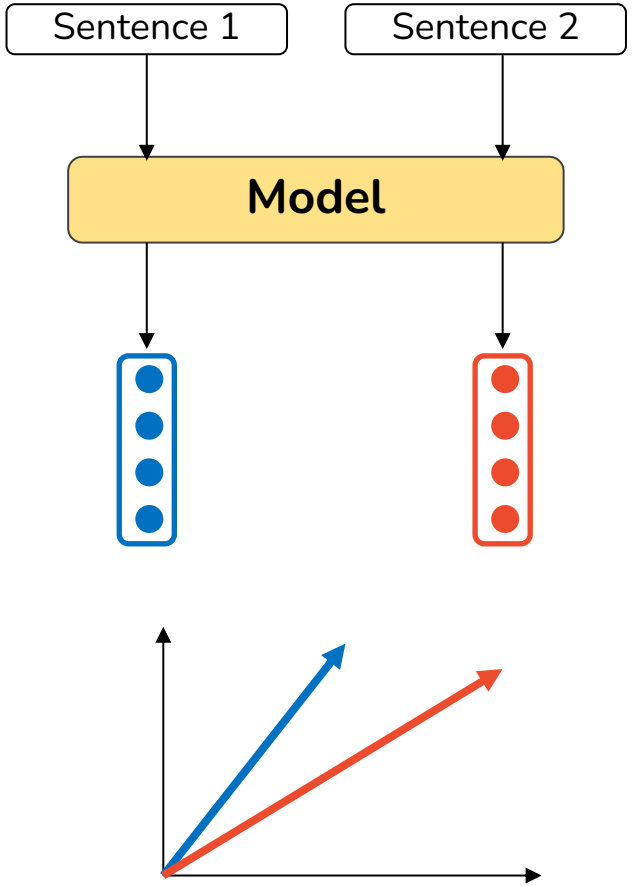


01.

SentenceBERT

What Is a Sentence Embedding?

- Sentence Embedding: a single fixed-size vector that represents the meaning of a whole sentence (or paragraph, or short document).
- Once each sentence is a vector, comparing meaning is just geometry (as with tf-idf):
- Cosine similarity (angle between vectors): close to 1 means very similar, near (or below) 0 means unrelated
- Or Euclidean distance: small distance means similar meaning



Can't We Just Use BERT for This?

Idea 1: Use vanilla BERT's [CLS] vector (or average its word vectors) as the sentence vector

- Weak for similarity. BERT's weights were only trained for masked-word prediction, never to make sentence vectors comparable
- On benchmarks: often worse than averaging plain GloVe vectors!

Idea 2: Feed both sentences into BERT together and let it judge similarity

- Accurate: BERT compares the two directly
- But it does not scale: we need to re-run BERT for every pair
- E.g. for 10k sentences → ~50M pairs → 65 hours!
- And real corpora have millions of sentences
- Cause: BERT mixes the two sentences inside the model, so no work can be reused

SentenceBERT (SBERT)

Nils Reimers

Fine-tuned BERT so its pooled vectors can be used for similarity.

- Run each sentence separately through BERT (not in pairs)
- Pool the word vectors into one fixed sentence vector (just average them, the default)
- That average is the sentence embedding

Why this is so much faster:

- You can pre-compute and store all the vectors ahead of time
- Comparing them later is just fast cosine similarity, no BERT needed

Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks

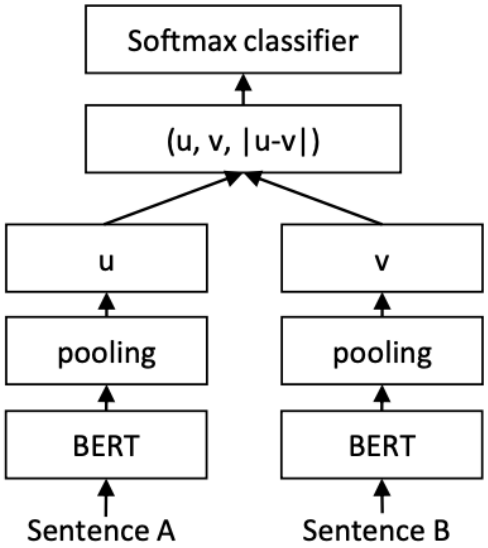
How SBERT Learns Good Sentence Vectors

The setup (a "siamese" network):

- Two copies of BERT that share the same weights
- Feed in pairs of sentences, get two sentence vectors
- Adjust the weights so that the vectors land in the right place

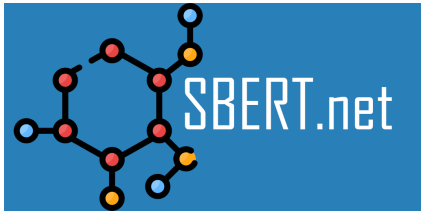
The training:

- Pairs of sentences labeled by their relationship (from Natural Language Inference data: do these agree, contradict, or neither?).
- After training, we throw away the training head and just keep the part that turns a sentence into a vector.



Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks

`sentence-transformers` library



- By UKPLab (TU Darmstadt), based on **`transformers`**
- Turning a sentence into a vector is a single function call
- Many ready-made models exist on the Hugging Face Hub, for example:
  - **`all-MiniLM-L6-v2`**: small and fast, a great default
  - **`all-mpnet-base-v2`**: higher quality, a bit slower
- multilingual and German models for non-English text
- The vectors are exactly what we need for search, clustering, and topic modeling
- Takeaway: you almost never train this yourself. You download a sentence-transformer and start embedding.

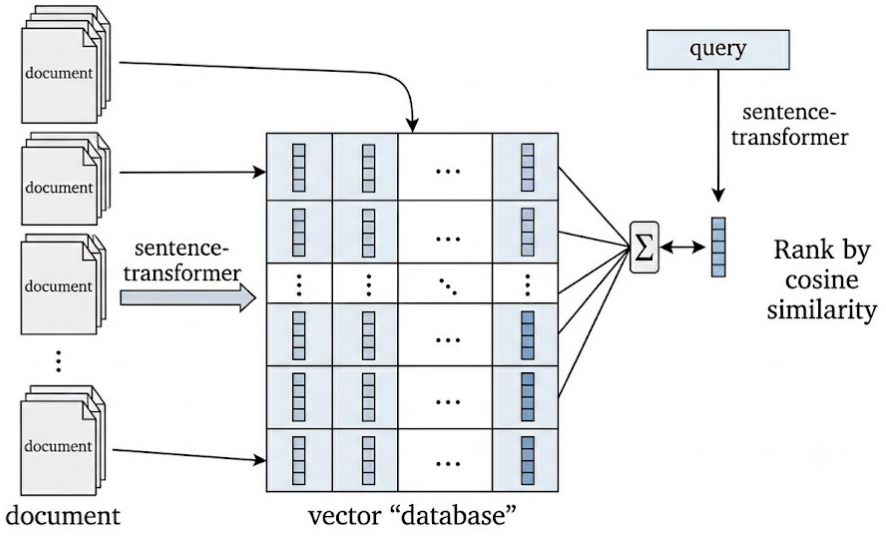
02.

Semantic Search

Semantic Search: Match Meaning, Not Words

- Keyword search matches shared words, so it misses the vocabulary mismatch case:
- Query "how to fix a flat tyre" vs. document "repairing a punctured wheel" → no match
- Semantic search: represent the query and every document as embeddings, then rank by similarity of meaning.

1. Offline (once): embed every document with a sentence-transformer, store the vectors in an index
2. At query time: embed the query, compare it to the stored document vectors (see later slide)
3. Rank by (cosine) similarity, return the top candidates



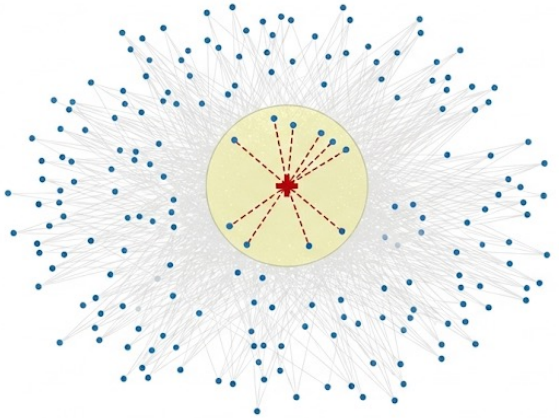
Scaling Up: Finding Neighbors Fast

Query vector must be compared to every stored document vector to find the closest ones.

- For a few thousand documents: easy, just compute all the cosines
- For millions of documents: comparing against every one is too slow

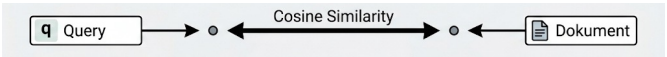
Approximate Nearest Neighbor (ANN) search solves this:

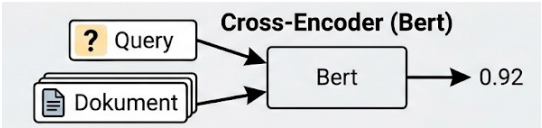
- Special index structures find the closest vectors without checking every single one
- Slightly approximate, but massively faster
- HNSW (Hierarchical Navigable Small World):
  a navigable graph you "hop" through to reach the nearest vectors quickly
- Libraries and vector databases implement this already

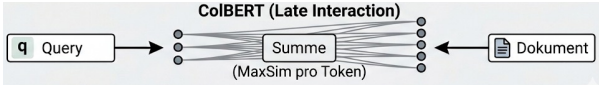


Three Ways to Compare Query and Document

	Bi-encoder (SBERT)	Cross-encoder	ColBERT
Input	each text **separately**	both texts **together**	each text **separately**
Stored as	**one vector** per text	nothing (no embedding)	**one vector per word**
Compare by	cosine of the two vectors	model outputs a score	best word matches (**MaxSim**)
Pre-compute?	**yes**	**no**	**yes** (per word)
Speed	**fast**, scales to millions	**slow**	medium
Accuracy	very good	**best** (sees both texts)	high
In short	fast & reusable, but a whole text is squeezed into one vector	most accurate, but re-run per pair, too slow for thousands of docs	per-word matching is more precise and still pre-computable, but stores many more vectors







Retrieve and Re-Rank

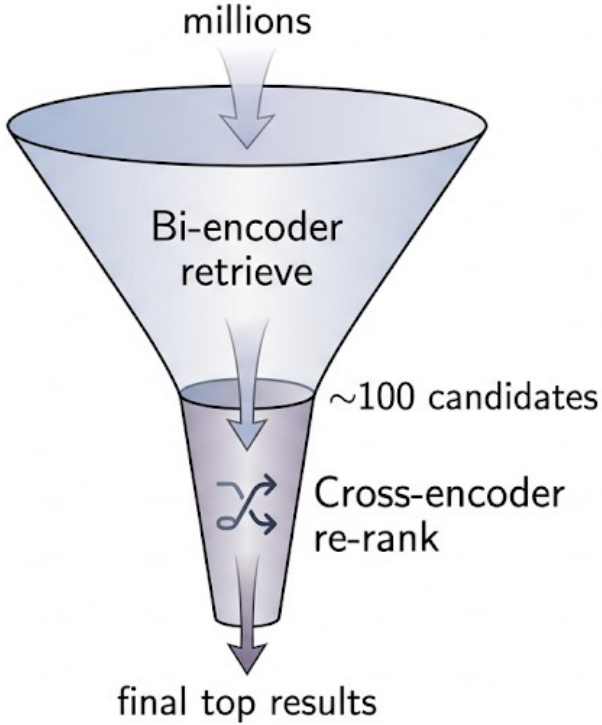
Stage 1 - Retrieve (bi-encoder):

- Fast search over the whole collection
- Pull back the top ~100 candidate documents

Stage 2 - Re-rank (cross-encoder):

- Carefully score just those 100 query-document pairs
- Reorder them so the best results rise to the top

Fast where you need scale, accurate where it matters.



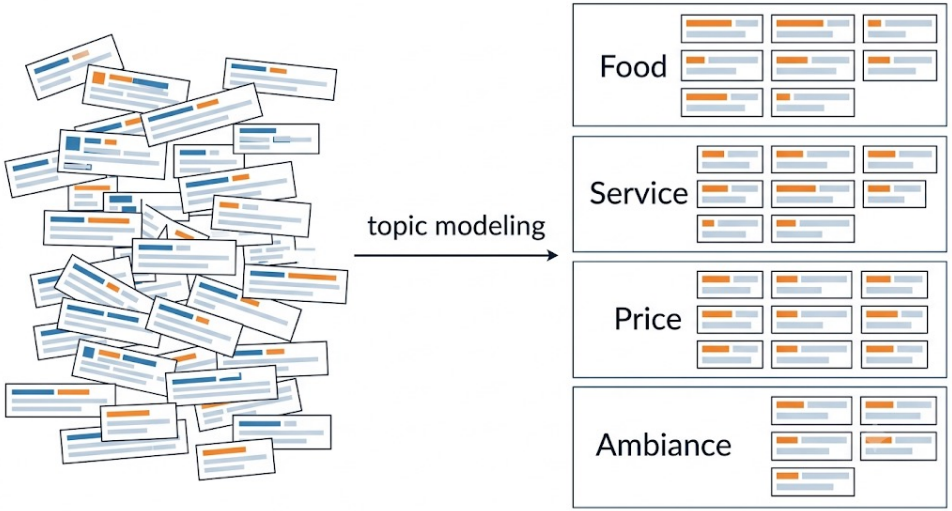
SBERT & Semantic Search in marimo

03.

BERTTopic

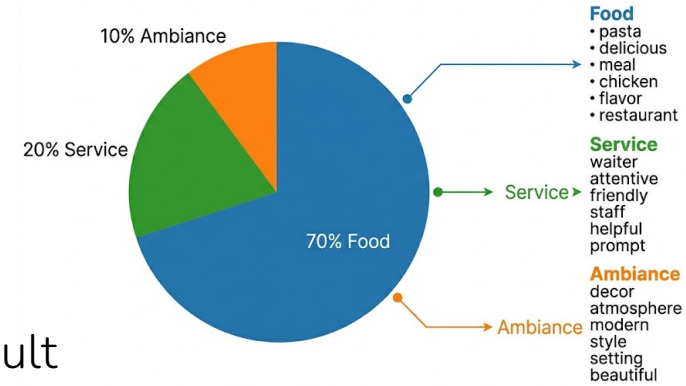
Topic Modeling

- Topic modeling: an unsupervised task that groups documents by theme and describes each theme with its most characteristic words.
- Input: a large collection of unlabeled text (reviews, survey answers, support tickets, ...)
- Output: a set of topics, each a cluster of documents plus a list of representative words
- No predefined categories, the themes are discovered from the data



The Classic Approach: LDA

- LDA (Latent Dirichlet Allocation): the long-standing standard for topic modeling.
- Probabilistic generative model: each document is a mixture of topics, each topic is a distribution over words
- Operates on word counts (bag-of-words): word order and meaning are ignored
- Number of topics K is a required input, set by the user
- Produces per-topic word lists; the topic label is assigned manually
- No notion of meaning: synonyms count as unrelated words
- Choosing K is non-trivial and strongly affects the result
- Resulting topics are often incoherent or hard to interpret



Interpreting Clusters: Human Annotation

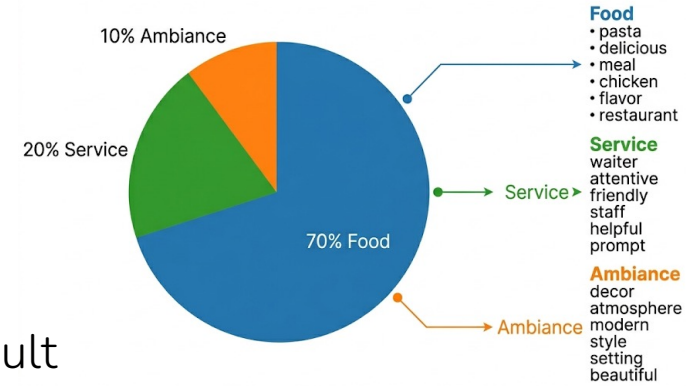
Topic ID	Top 8 Words	Human-Interpreted Label
Topic 0	customers, rude, great, food, management, people, work, fast	Customer Interactions & Pace
Topic 1	work, life, company, employees, balance, cons, management, think	Work-Life Balance & Company Culture
Topic 2	shifts, experience, scheduling, late, little, coworkers, work, opportunities	Scheduling & Coworker Issues
Topic 3	time, work, hours, management, don, hard, job, schedule	?
Topic 4	management, pay, low, hours, poor, bad, work, lack	?
Topic 5	hours, team, work, store, members, managers, help, manager	?

Interpreting Clusters: Human Annotation

Topic ID	Top 8 Words	Human-Interpreted Label
Topic 0	customers, rude, great, food, management, people, work, fast	Customer Interactions & Pace
Topic 1	work, life, company, employees, balance, cons, management, think	Work-Life Balance & Company Culture
Topic 2	shifts, experience, scheduling, late, little, coworkers, work, opportunities	Scheduling & Coworker Issues
Topic 3	time, work, hours, management, don, hard, job, schedule	Workload & Hours
Topic 4	management, pay, low, hours, poor, bad, work, lack	Compensation & Poor Management
Topic 5	hours, team, work, store, members, managers, help, manager	In-Store Management & Team Dynamics

The Classic Approach: LDA

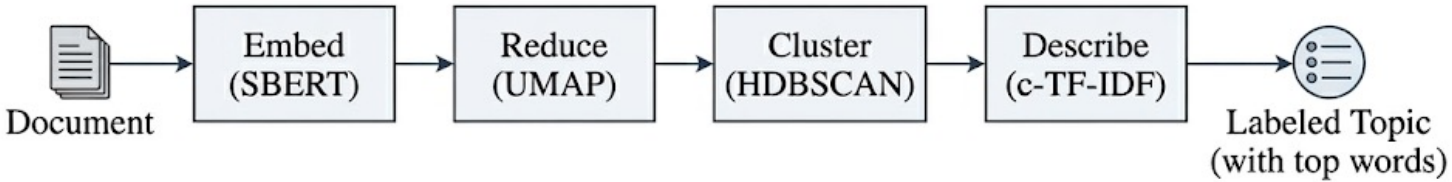
- LDA (Latent Dirichlet Allocation): the long-standing standard for topic modeling.
- Probabilistic generative model: each document is a mixture of topics, each topic is a distribution over words
- Operates on word counts (bag-of-words): word order and meaning are ignored
- Number of topics K is a required input, set by the user
- Produces per-topic word lists; the topic label is assigned manually
- No notion of meaning: synonyms count as unrelated words
- Choosing K is non-trivial and strongly affects the result
- Resulting topics are often incoherent or hard to interpret



The Modern Approach: BERTopic

- BERTopic: topic modeling by clustering sentence embeddings, so topics are based on meaning rather than word counts.
- Four-step pipeline, each step independently swappable:

1. Embed each document with a sentence-transformer
2. Reduce the embeddings to a few dimensions (UMAP) so clustering is reliable
3. Cluster the reduced vectors (HDBSCAN) into groups of similar meaning
4. Describe each cluster by its most distinctive words (c-TF-IDF)



How BERTopic Names a Topic: c-TF-IDF

After clustering, each topic is a group of documents with no label yet.

Class-based TF-IDF (c-TF-IDF):

- Treat all documents in one cluster as a single document
- Score words by how frequent they are in this cluster but rare in the others
- The top-scoring words form the topic description
- Standard TF-IDF compares words across documents; c-TF-IDF compares them across clusters
- Common words ("the", "and") score low; cluster-specific words ("delivery", "shipping") score high and characterize the topic

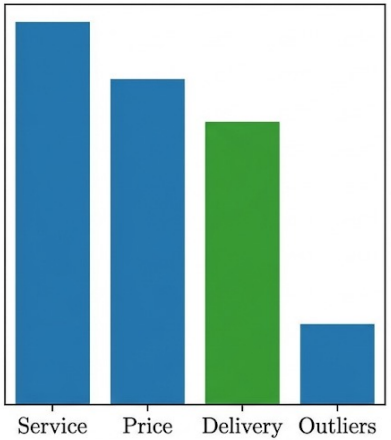
Why BERTopic over LDA

	LDA	BERTopic
Basis	word counts	**meaning** (embeddings)
Synonyms	treated as unrelated	grouped together
Number of topics	**must choose K**	found automatically
Outliers	forced into a topic	can stay **unassigned**
Topics	often hard to read	usually **more coherent**

- Topics are based on meaning, so paraphrases and synonyms group together
- HDBSCAN determines the number of topics and can leave noisy documents unassigned
- Modular: step 1 can be swapped for any other embedding model e.g. for non-English text

Applications

- Customer feedback: identify the most frequent complaints to prioritize (product, HR)
- Product reviews: surface recurring themes (battery life, shipping, ease of use)
- Research: analyze large text collections for themes, bias, or change over time
- Because HDBSCAN allows outliers, off-topic documents are collected in a separate group rather than distorting the real topics.




```
tracking
delay
received
packaging
ontime

The tracking number said it
would be here yesterday, but it
is dlayed. Still not received.


Fast delivery, but the packaging
was damaged. Item seems okay,
but frustrating.
```

BERTopic in marimo